Validation

Signup form accumulates all field errors

Season 6 · Functional · Intermediate

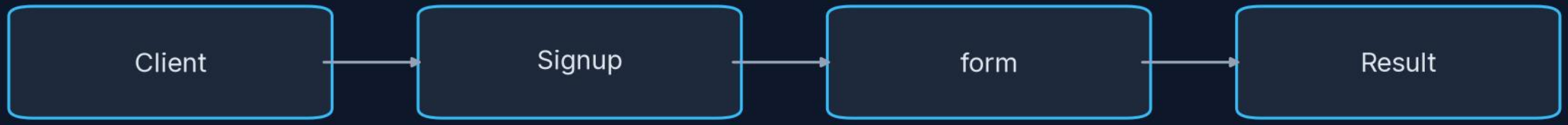
Today's Hook

- Problem: Signup form accumulates all field errors
- Naive pain cue: caught=email
- We will refactor to Validation with a runnable Demo.
- Failure path is locked by ProblemTest.

The Problem

- Scenario: Signup form accumulates all field errors
- Constraints: solve «Signup form accumulates all field errors» with Validation — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Signup form accumulates all field errors
- Success looks like: Valid[value=welcome a@b.c]

Structure



Validation: Signup form accumulates all field errors

Real-World Scenario

- Running example: Signup form accumulates all field errors
- Key types: Validation, Valid, Invalid, Signup
- Happy-path stdout cue: Valid[value=welcome a@b.c]
- Failure-path stdout cue: caught=email

Naive Code

```java
package com.buildwithwaqar.patterns.functional.validation.problem;

import java.util.*;

public final class NaiveSignup {
  public static String signup(String email, int age){
    if(!email.contains("@")) throw new IllegalArgumentException("email");
    if(age<18) throw new IllegalArgumentException("age");
    return "ok:"+email;
  }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Signup form accumulates all field errors
- Without Validation, call sites absorb the wrong details.
- Failure mode to show on camera: caught=email
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.functional.validation.problem;

import java.util.*;
import java.util.function.*;

sealed interface Validation<E,A> permits Valid, Invalid {
  boolean isValid(); A get(); List<E> errors();
  default <B> Validation<E,B> map(Function<A,B> f){ return isValid()? new Valid<>(f.apply(get())): new Invalid<>(errors()); }
  default <B> Validation<E,B> zipWith(Validation<E,B> other, BiFunction<A,B,?> ignored){ throw new UnsupportedOperationException(); }
}
record Valid<E,A>(A value) implements Validation<E,A> {
  public boolean isValid(){return true;} public A get(){return value;} public List<E> errors(){return List.of();}
}
record Invalid<E,A>(List<E> errs) implements Validation<E,A> {
  public boolean isValid(){return false;} public A get(){throw new IllegalStateException();} public List<E> errors(){return errs;}
}
final class Signup {
  static Validation<String,String> email(String e){ return e!=null && e.contains("@")? new Valid<>(e): new Invalid<>(List.of("email has no @")); }
  static Validation<String,Integer> age(int a){ return a>=18? new Valid<>(a): new Invalid<>(List.of("age under 18")); }
  static Validation<String,String> form(String e, int a){
    var ve=email(e); var va=age(a);
    if (ve.isValid() && va.isValid()) return new Valid<>("welcome "+ve.get());
    List<String> errs=new ArrayList<>(); errs.addAll(ve.errors()); errs.addAll(va.errors());
    return new Invalid<>(errs);
  }
```

Solution (2)

```
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.validation.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Validation: signup accumulates errors ===");
    System.out.println("--- naive ---");
try { System.out.println(NaiveSignup.signup("bad", 10)); } catch(Exception e){ System.out.println("caught="+e.getMessage()); }
System.out.println("--- validation ---");
System.out.println(Signup.form("a@b.c", 20));
System.out.println(Signup.form("bad", 10));
  }
}
```

Walkthrough Output

```
=== Validation: signup accumulates errors ===
--- naive ---
caught=email
--- validation ---
Valid[value=welcome a@b.c]
Invalid[errs=[email has no @, age under 18]]
```

Step-by-Step Trace

- Match stdout to stages — look for: Valid[value=welcome a@b.c]
- Then the injected failure: caught=email
- Re-run: java -cp ... 45-validation ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```
package com.buildwithwaqar.patterns.functional.validation.problem;

import java.util.*;
import java.util.function.*;

sealed interface Validation<E,A> permits Valid, Invalid {
  boolean isValid(); A get(); List<E> errors();
  default <B> Validation<E,B> map(Function<A,B> f){ return isValid()? new Valid<>(f.apply(get())): new Invalid<>(errors()); }
  default <B> Validation<E,B> zipWith(Validation<E,B> other, BiFunction<A,B,?> ignored){ throw new UnsupportedOperationException(); }
}
record Valid<E,A>(A value) implements Validation<E,A> {
  public boolean isValid(){return true;} public A get(){return value;} public List<E> errors(){return List.of();}
}
record Invalid<E,A>(List<E> errs) implements Validation<E,A> {
  public boolean isValid(){return false;} public A get(){throw new IllegalStateException();} public List<E> errors(){return errs;}
}
final class Signup {
  static Validation<String,String> email(String e){ return e!=null && e.contains("@")? new Valid<>(e): new Invalid<>(List.of("email has no @")); }
  static Validation<String,Integer> age(int a){ return a>=18? new Valid<>(a): new Invalid<>(List.of("age under 18")); }
  static Validation<String,String> form(String e, int a){
```

Trade-offs

- Use Validation when «Signup form accumulates all field errors» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Validation in product language.

Related Patterns

- Name the pattern usually confused with Validation and separate the forces.
- Sketch the roles for «Signup form accumulates all field errors» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Signup form accumulates all field errors
- Naive pain → Validation boundary → Demo stdout.
- Remember the failure cue: caught=email
- Break a ProblemTest assertion, watch it fail, restore it.